The allegory axioms, and what defines them in the Frobenius calculus

§1. Notation

Traditionaly Adjunction F -| G is defined as $F(x) \leq y$ iff $x \leq G(y)$, and you derive laws like this:

$$\begin{aligned} F(x) \leq F(x) &\implies x \leq G(F(x)) &\implies F(x) \leq F(G(F(x))) \\ G(y) \leq G(y) &\implies F(G(y)) \leq y &\implies G(F(G(y))) \leq G(y) \end{aligned}$$

By define $X : * \mapsto x$, $Y : * \mapsto y$ and use diagram order, application can be replaced by composition: $XF \leq Y$ iff $X \leq YG$

- write F as $/$, G as $\backslash$ (1a)
- render $a \leq b$ as $\boxed{\frac{a}{b}}$
- **rewrite** $X \leq XF$ as $1 \leq F$

$$\boxed{\frac{X/}{Y}} = \boxed{\frac{X}{Y\backslash}} \qquad \text{adj}$$

$$\boxed{\frac{X/}{X/}} \xRightarrow{\{\text{adj}\}} \boxed{\frac{X}{X/\backslash}} \overset{\{\text{rewrite}\}}{=\!=\!=} \underset{\text{unit}}{\boxed{\frac{*}{/\backslash}}} \begin{cases} \xRightarrow{\{\text{put } \backslash \text{ on left}\}} \boxed{\frac{\backslash}{\backslash/\backslash}} \\ \xRightarrow{\{\text{put } / \text{ on right}\}} \boxed{\frac{/}{/\backslash/}} \end{cases}$$

$$\boxed{\frac{Y\backslash}{Y\backslash}} \xRightarrow{\{\text{adj}\}} \boxed{\frac{Y\backslash/}{Y}} \overset{\{\text{rewrite}\}}{=\!=\!=} \underset{\text{counit}}{\boxed{\frac{\backslash/}{*}}} \begin{cases} \xRightarrow{\{\text{put } / \text{ on left}\}} \boxed{\frac{/\backslash/}{/}} \\ \xRightarrow{\{\text{put } \backslash \text{ on right}\}} \boxed{\frac{\backslash/\backslash}{\backslash}} \end{cases}$$

now we have $/\backslash/ = /$ and $\backslash/\backslash = \backslash$, and you just discovered string diagrams.

§1.1. Every adjunction in Rel

Both halves of a row are written as sections — the operator stays and its missing argument is the gap: `·S` is `R ↦ R S`, `/S` is `T ↦ T/S`, `S\` is `Y ↦ S\Y`. Composition is juxtaposition, so the dot is written only in the rows where nothing else marks the gap.

F ⊣ G	F's type	monad FG	𝟙⊑FG	GF⊑𝟙	F preserves ∪	G preserves ∩	FGF=F	GFG=G
◁ ⊣ ▷	A → A⊗A	◁▷=𝟙	𝟙⊑◁▷	▷◁⊑𝟙	(R∪S)◁= R◁∪S◁	(R∩S)▷= R▷∩S▷	◁▷◁=◁	▷◁▷=▷
⊸ ⊣ ⟜	A → 𝕀	⊸⟜=⊤	𝟙⊑⊸⟜	⟜⊸⊑𝟙	(R∪S)⊸= R⊸∪S⊸	(R∩S)⟜= R⟜∩S⟜	⊸⟜⊸=⊸	⟜⊸⟜=⟜
° ⊣ °	(A⇸B) → (B⇸A)	°°=𝟙	R=R°°	R=R°°	(R∪S)°= R°∪S°	(R∩S)°= R°∩S°	R°°°=R°	R°°°=R°
⟜◁ ⊣ ▷⊸	(X⊗A → Y) → (X → Y⊗A)	𝟙	(⟜◁⊗𝟙) (𝟙⊗▷⊸)=𝟙	(𝟙⊗⟜◁) (▷⊸⊗𝟙)=𝟙	=	=	=	=
Δ ⊣ ∩	(A⇸B) → (A⇸B)^2	R↦R∩R=R	R⊑R∩R	R∩S⊑R	—	—	R∩R=R	R∩R=R
∪ ⊣ Δ	(A⇸B)^2 → (A⇸B)	(R,S)↦ (R∪S,R∪S)	R⊑R∪S	R∪R⊑R	—	—	R∪R=R	R∪R=R
⊥ ⊣ !	{*} → (A⇸B)	—	—	⊥⊑R	—	—	—	—
·S ⊣ /S	(A⇸B) → (A⇸C)	S/S	R⊑(RS)/S	(T/S)S⊑T	(R∪T)S= RS∪TS	(R∩T)/S= R/S∩T/S	((RS)/S)S =RS	((T/S)S)/S =T/S
S· ⊣ S\	(B⇸C) → (A⇸C)	S\S	R⊑S\(SR)	S(S\T)⊑T	S(R∪T)= SR∪ST	S\(R∩T)= S\R∩S\T	S(S\(SR)) =SR	S\(S(S\T)) =S\T
R∩ ⊣ R⇒	(A⇸B) → (A⇸B)	X↦R⇒(X∩R)	X⊑R⇒(X∩R)	R∩(R⇒Y)⊑Y	R∩(X∪Y)= (R∩X)∪(R∩Y)	R⇒(X∩Y)= (R⇒X)∩(R⇒Y)	R∩(R⇒(X∩R)) =X∩R	R⇒(R∩(R⇒Y)) =R⇒Y
𝓓 ⊣ ·⊤	(A⇸B) → Cor A	R↦(𝓓R)⊤	R⊑(𝓓R)⊤	𝓓(A⊤)⊑A	𝓓(R∪S)= 𝓓R∪𝓓S	(A∩B)⊤= A⊤∩B⊤	𝓓((𝓓R)⊤) =𝓓R	(𝓓(A⊤))⊤ =A⊤
𝓡 ⊣ ⊤·	(A⇸B) → Cor B	R↦⊤(𝓡R)	R⊑⊤(𝓡R)	𝓡(⊤A)⊑A	𝓡(R∪S)= 𝓡R∪𝓡S	⊤(A∩B)= ⊤A∩⊤B	𝓡(⊤(𝓡R)) =𝓡R	⊤(𝓡(⊤A)) =⊤A
·f ⊣ ·f°	(A⇸B) → (A⇸C)	ff°	𝟙⊑ff°	f°f⊑𝟙	(R∪S)f= Rf∪Sf	(R∩S)f°= Rf°∩Sf°	ff°f=f	f°ff°=f°
f°· ⊣ f·	(A⇸C) → (B⇸C)	ff°	𝟙⊑ff°	f°f⊑𝟙	f°(X∪Y)= f°X∪f°Y	f(X∩Y)= fX∩fY	ff°f=f	f°ff°=f°
i ⊣ E	Map ↪ Rel	P	{·}:A⇸P A	∋:P B⇸B	—	—	Λ(∋)=𝟙	Λ(R)∋=R

(1.1a)

- **parameters.** `S : B → C` in the `·S` row and `A → B` in the `S·` one; the map `f` likewise, `B → C` in the `·f` row and `A → B` in the `f°·` one. `Cor A` is the poset of coreflexives on `A`: the `A` and `B` of the 𝓓 and 𝓡 rows are coreflexives, not objects.
- **the ⟜◁ row** bends one wire from the input side to the output side — §1.2 below names that operator `R^` and draws it. It is Rel's compact closure, every object its own dual, and the reason ⊗ gives no exponential: Rel's categorical product is +, not ⊗. Both halves are order-isomorphisms, so the last four columns are equalities with nothing to write in them; `° ⊣ °` is the table's other row of that kind.
- **the monad FG** is a 1-cell `M : A → A` with `𝟙 ⊑ M` and `MM ⊑ M`, in Rel a preorder on `A`, equivalently `M = M/M` (B&dM Ex 4.49(i)). row lies between them: `ff°` is the kernel of `f`, `a (ff°) a' ⟺ f a = f a'`, and `S/S` is `x (S/S) y ⟺ S[y] ⊆ S[x]`. Only `·S` and `S·` are inexact — the operator monad there is `X ↦ (XS)/S`, and `X(S/S) ⊑ (XS)/S` with equality when `X` is a map.
- **𝟙⊑FG is that monad's unit**, hidden because the row names the two factors rather than the composite: in diagram order `FG` is what is usually written `G∘F`. The multiplication and the comultiplication need no columns of their own, being got by composing the two that are there with `F` and `G` —

  `FGFG ⊑ F𝟙G = FG` the counit `GF⊑𝟙`, with `F` before it and `G` after
  `GF = G𝟙F ⊑ G(FG)F` the unit `𝟙⊑FG`, with `G` before it and `F` after

  and neither do the coherence laws: hom-posets are thin, so every diagram commutes on its own.
- **⊸⟜ = ⊤**, for any `R : A → B`:

  `R = R 𝟙_B ⊑ R(⊸_B ⟜_B) = (R⊸_B) ⟜_B ⊑ ⊸_A ⟜_B` the unit of `⊸ ⊣ ⟜`, then the laxness of `⊸`; both pictured below

  Every arrow `A → B` therefore sits under `⊸_A⟜_B`, the `⊤` the `∩` section below takes as the unit of the meet. In Rel `⊸` discards whatever it is handed and `⟜` creates anything, so the composite relates every `a` to every `b`: as a subset, all of `A×B` — the cartesian product of the underlying SETS, the ambient `Rel(A,B) = 2^(A×B)`, and not a categorical product in Rel.

§1.2. Composing adjunctions

One more adjunction first, which §1.1 cannot hold because it is ANTITONE — its left adjoint turns joins into meets, so that table's `F` preserves `∪` column would read the wrong way; its type is contravariant, `F : (B⇸C)^op → (A⇸B)`, so the `F`'s type column cannot be written in §1.1's form either; and BOTH composites are inflationary, `𝟙⊑FG` and `𝟙⊑GF`, so the `GF⊑𝟙` column has no entry for it at all. That last is the signature of an antitone Galois connection — two closure operators rather than a closure and an interior — and the table's own **both units** cell is it:

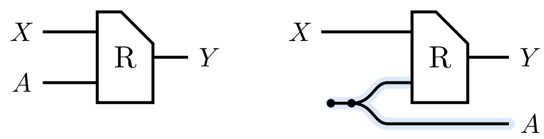
`F ⊣ G`	**the law**	**both units**
`R/ ⊣ \R`	`T⊑R/S` iff `S⊑T\R`	`S⊑(R/S)\R`, `T⊑R/(T\R)`

(1.2a)

Two rows of §1.1 composed are a third adjunction, and its right adjoint read in the two orders is a law. Row first, column second; right adjoints compose the other way round, which is where every reversal in the table comes from.

The table's `⊸◁` row and column need a name for that row's left adjoint, and §1.1 gave it none: write `R^` for the operator listed there, `R ↦ (𝟙⊗⊸◁)(R⊗𝟙)`.

`R : X⊗A → Y`   `R^ = (𝟙⊗⊸◁)(R⊗𝟙) : X → Y⊗A` (1.2b)



(1.2c)

left `R`, right `R^`: the pale strand is everything the bend adds, and the box and the `X`, `Y` wires are where they were.

In `Rel` the two are one set of triples split two ways — the bend moves the `A` coordinate from the input side to the output side:

`x R^ (y,a) ⟺ (x,a) R y` (1.2d)

Read `Rel(I,J)` as an `I×J` boolean matrix — Freyd's §2.111, which defines reciprocation entrywise as `j R° i = i R j` — and `R^` is the smallest of three degrees of bending, all of the same `R : X⊗A → Y`:

	what moves	**type**	**as a matrix**
`R^`	one index, to the output side	`X → Y⊗A`	rows re-indexed from `(x,a)` to `x`, columns from `y` to `(y,a)`
`R°`	the input index goes out and the output index comes in drawn in the ° section below	`Y → X⊗A`	the transpose
`⌜R⌝`	everything, to the output side — the **name** of `R`	`𝕀 → Y⊗A⊗X`	the whole matrix flattened into one row

(1.2e)

All three carry the same entries; only the split of the indices into row and column changes. That is what a matrix transpose always was — moving an index from the domain side to the codomain side — and compact closure is the structure that permits the move: `Rel` is self-dual, `A* = A`, so there is no up/down index distinction to keep track of.

Of the three, only `^` is this note's own, the half bend having no settled symbol. `⌜R⌝` and its mirror the **coname** `⌞R⌟ : Y⊗A⊗X → 𝕀` are the literature's. Neither is the transpose `Λ` of B&dM, which is Freyd's §2.421 `R/S = Λ(R)Λ°(S)` as well: that one sends `R : X⊗A → Y` to a MAP `Λ R : X⊗A → P Y`, and needs a power allegory rather than compact closure.

(1.2f)

	·T	T·	·g	g°·	T∩	°	⊶◁	Δ
·S	R/(ST)= (R/T)/S	T\(R/S)= (T\R)/S	R/(Sg)= (Rg°)/S	g(R/S)= (gR)/S	—	(Y/S)°= S°\(Y°)	(RS)^= R^(S⊗𝟙)	$(T_1\cap T_2)/S= T_1/S\cap T_2/S$
S·	S\(R/T)= (S\R)/T	(TS)\R= S\(T\R)	S\(Rg°)= (S\R)g°	(g°S)\R= S\(gR)	—	(S\Y)°= Y°/S°	((S⊗𝟙)R)^= S R^	$S\backslash(T_1\cap T_2)= S\backslash T_1\cap S\backslash T_2$
·f	R/(fT)= (R/T)f°	T\(Rf°)= (T\R)f°	(fg)°=g°f°	—	—	(fY)°=Y°f°	—	$(T_1\cap T_2)f^\circ= T_1f^\circ\cap T_2f^\circ$
f°·	f(R/T)= (fR)/T	(Tf°)\R= f(T\R)	—	(fg)°=g°f°	—	(fY)°=Y°f°	—	$f(T_1\cap T_2)= fT_1\cap fT_2$
R∩	—	—	—	—	(R∩T)⇒Y= R⇒(T⇒Y)	(R⇒Y)°= R°⇒(Y°)	—	$R\Rightarrow(T_1\cap T_2)= (R\Rightarrow T_1)\cap(R\Rightarrow T_2)$
°	(Y/T)°= T°\(Y°)	(T\Y)°= Y°/T°	(gY)°=Y°g°	(gY)°=Y°g°	(T⇒Y)°= T°⇒(Y°)	Y°°=Y	(R^)°= (R°⊗𝟙)(𝟙⊗▷⊸) both bends: the ° section's display	$(T_1\cap T_2)^\circ= T_1^\circ\cap T_2^\circ$
⊶◁	(RT)^= R^(T⊗𝟙)	((T⊗𝟙)R)^= T R^	—	—	—	(R^)°= (R°⊗𝟙)(𝟙⊗▷⊸) both bends: the ° section's display	—	—
∪	$(X_1\cup X_2)T= X_1T\cup X_2T$	$T(X_1\cup X_2)= TX_1\cup TX_2$	$(X_1\cup X_2)g= X_1g\cup X_2g$	$g^\circ(X_1\cup X_2)= g^\circ X_1\cup g^\circ X_2$	$T\cap(X_1\cup X_2)= (T\cap X_1)\cup(T\cap X_2)$	$(X_1\cup X_2)^\circ= X_1^\circ\cup X_2^\circ$	—	—
⊥	⊥T=⊥	T⊥=⊥	⊥g=⊥	g°⊥=⊥	T∩⊥=⊥	⊥°=⊥	—	—

§1.3. Adjoint triples

Beyond ∪ ⊣ Δ ⊣ ∩, which §1.1 already carries as two rows, the table holds one adjoint triple with content: ∃_f ⊣ f* ⊣ ∀_f along a map f, once on each side of the composite. f : A → B throughout this subsection.

(1.3a)

$$\cdot f \dashv \cdot f^\circ \dashv /f^\circ$$
$$f^\circ\cdot \dashv f\cdot \dashv f\backslash$$

The first two links of each chain are the map shunting rules, and the third is §1.1's division row read at a chosen divisor: ·S ⊣ /S at S := f°, and S· ⊣ S\ at S := f.

The third link is not the first over again. For a map f, /f collapses to ·f°; being a map is exactly what that collapse spends R/(f S) = (R/S) f°. — f° is not a map, so /f° does not collapse in turn, and the three operators stay distinct. On Rel(𝕀, A) = P A they are the image triple:

(1.3b)

operator	acts by	name
·f	S ↦ f[S]	direct image, ∃_f
·f°	T ↦ f^{-1}[T]	inverse image, f*
/f°	S ↦ {b : f^{-1}(b) ⊆ S}	∀_f

§1.1's 𝓓 ⊣ ·T is this same chain along the projection A⊗B → A, read through Rel(A, B) = P(A⊗B): 𝓓R = {(a,a) : ∃b. aRb}, AT is the relation that ignores b altogether, and the third link is R ↦ 𝟙 ∩ R/T = {(a,a) : ∀b. aRb}.

(1.3c)

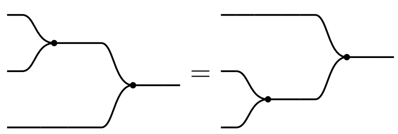
$$\mathcal{D} \dashv \cdot T \dashv \mathbb{1}\cap\cdot/T$$

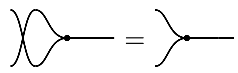
Three is where it stops, and one map breaks both ends. Take A = $\{a_1,a_2\}$ and B = {b}: $f[\{a_1\}] \cap f[\{a_2\}] = \{b\}$ while $f[\{a_1\} \cap \{a_2\}] = \varnothing$, so ·f does not preserve meets and has no left adjoint, and the same two subsets give $\forall\_f(\{a_1\} \cup \{a_2\}) = \{b\}$ against $\forall\_f\{a_1\} \cup \forall\_f\{a_2\} = \varnothing$, so /f° does not preserve joins and has no right adjoint. A monic f restores the binary case and no more — Tf still reaches only im f, and ∀_f⊥ = B ∖ im f is still not ⊥. The chain extends only when f° is a map as well, and then /f° = ·f°° = ·f and it repeats forever. For an S with neither S nor S° a map there is no triple at all: ·S ⊣ /S is two links and stops.

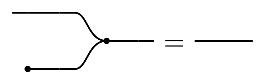
The rows that chain forever say nothing by it: ° is an order-isomorphism of hom-posets, so it is adjoint to itself on both sides and ° ⊣ ° ⊣ ° has no end, and §1.1's other row of that kind, ⊶◁ ⊣ ▷⊸, goes the same way.

§2. Relations

> Rel is a poset-enriched category with (⊗, °) where ⊗ is commutative cartisian product, and converse `°` ⊣ `°`, and
>
> C: `(▷ : A ⊗ A → A, ⟜ : 𝕀 → A)` a commutative monoid with `C°` ⊣ `C`. We write `C°` as `(◁ : A → A ⊗ A, ⊸ : A → 𝕀)` (2a)


(2b)

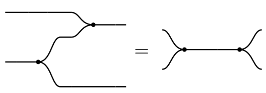


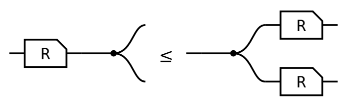


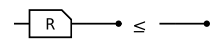
▷ associative

▷ commutative

⟜ is its unit





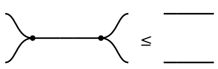


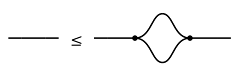
Frobenius, one half — the other is its °

`R◁ ≤ ◁(R⊗R)`

`R⊸ ≤ ⊸`

§2.1. ∀ object A, `(A, ◁, ⊸) ⊣ (A, ▷, ⟜)`


(2.1a)



`▷ ◁ ≤ 𝟙`

`𝟙 ≤ ◁ ▷`

`⟜ ⊸ ≤ 𝟙`

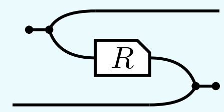
`𝟙 ≤ ⊸ ⟜`

The last of these is the only one that makes a picture **bigger**, and it is worth a name: **a wire is below the cut wire**. In `Rel` it reads `{(a, a)} ⊆ A × A` — cut a wire and its two ends stop having to agree, so cutting can only add pairs. It is the one weakening this calculus gives away for free.

§3. $\circ : \mathcal{C}^{\mathsf{op}} \longrightarrow \mathcal{C}$ is a 2 functor

(3a)

° is primitive, part of the data the first section lists. It turns both of `R`'s wires round, and the Frobenius structure DRAWS that — the picture and the formula below are `R°`, not its definition:



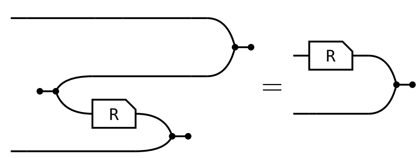
`R° = (⟜◁ ⊗ 𝟙) (𝟙 ⊗ R ⊗ 𝟙) (𝟙 ⊗ ▷⊸)`

where `⟜◁ : 𝕀 → a ⊗ a` opens a pair of wires out of nothing and `▷⊸ : b ⊗ b → 𝕀` closes one, so the input of `R°` is where the output of `R` was. Taking that formula as the definition would now be circular: ◁ and ⊸ are ▷° and ⟜°. The laws of ° are that it is a contravariant 2-functor `° : 𝒞ᵒᵖ → 𝒞`:

(i) `𝟙° = 𝟙` (ii) `(R S)° = S° R°` (iii) `(R ⊗ S)° = R° ⊗ S°` (iv) `R ≤ S` implies `R° ≤ S°`

§3.1. The slide

The one rule (ii) and (iii) use, and each of them uses it twice. A converse facing a merge on the lower strand is the box itself, upright, on the upper one:



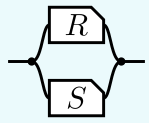
(3.1a)

`R°` is DEFINED as the bending of `(R ⊗ 𝟙) ▷⊸`, so the slide claims only that unbending it gives that back — and unbending undoes bending for every arrow. That is the snake above with a passenger: the `⟜◁` bends the `a` strand down and the `▷⊸` brings it back up, while the `b` strand rides through untouched. Nothing else is spent below.

§4. ∩ is a commutative idempotent monoid on every hom-set

(4a)

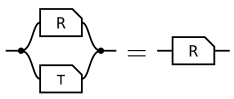
The **meet** of `R, S : a → b`, the paper's **convolution**, is `R ∩ S := ◁ (R ⊗ S) ▷` — copy the input, run `R` and `S` on the two copies, merge the results — so what comes out is what both of them do.



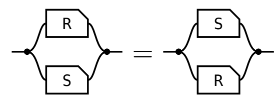
transcribed: a definition has no statement to export

On every hom-set it is associative, commutative and idempotent, with unit the maximal arrow `T = ⊸ ⟜` (the paper's Lemma 4.11).

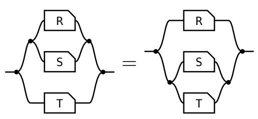
(4b)



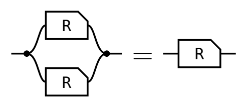
unit: one half of T per end — the merge's unit law absorbs the ⟜, the copy's counit law the ⊸



commutative: σ crosses R ⊗ S by naturality and is absorbed by cocommutativity and commutativity



associative: coassociativity and associativity; ⊗ re-brackets for nothing, being strict here

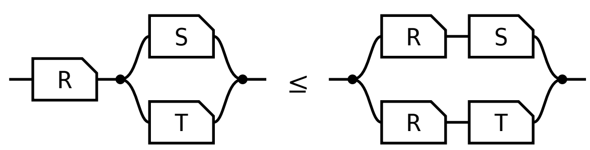


idempotent: the one that is not bookkeeping — the lax copy law is the whole of it, worked in allegory2

So ≤ is the order this monoid induces. `R ∩ S ≤ R` comes from the unit, and idempotency turns anything under both `S` and `T` into something under `S ∩ T`, since `R = R ∩ R ≤ S ∩ T`.

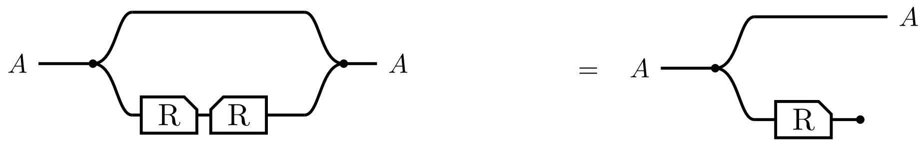
And one law relating ∩ to composition, which is **not** an equation:

(4c)

semi-distributivity, and what supplies it	**picture**
`R (S ∩ T) ⊑ R S ∩ R T` — the lax copy law. Equality exactly when R is single valued: the Maps section's `F (R ∩ S) = F R ∩ F S`.	

§5. Domain and range

> The **domain** `Dom R ≙ 𝟙 ∩ R R°` and the **range** `Ran R ≙ Dom R°`. (5a)



(5b)

▷ lands the return leg back on the value ◁ handed out,
so `R°` ▷ cuts to ⊸ — Frobenius

Running `R` and throwing the result away leaves only the fact that `R` could fire, and `Ran R` is the same picture with the box mirrored. In `Rel` both steps are `{(a,a) : ∃b. a R b}`.

(5c)

`Dom R ≙ 𝟙 ∩ R R°`
`Dom R ⊑ A` ⟺ `R ⊑ A R`, for `A` coreflexive
`Dom (R S) ⊑ Dom R`
`Dom (R ∩ S) = 𝟙 ∩ S R°`
`R` entire ⟺ `Dom R = 𝟙` ⟺ `𝟙 ⊑ R R°`
`R` simple ⟺ `R° R ⊑ 𝟙`
`R` a map ⟺ `R` entire and simple
`R, S` entire ⟹ `R S` entire — likewise simple, likewise maps
`R S` entire ⟹ `R` entire

§5.1. Sliding the discard

`Dom (R S) ⊑ Dom R`, and a single glyph for `Dom` would have nothing to slide: with the box and the discard drawn apart, the law is one dot walking back along the lower strand.

(5.1a)

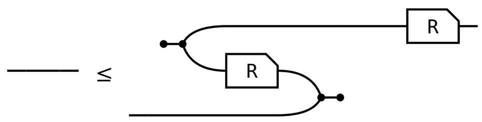
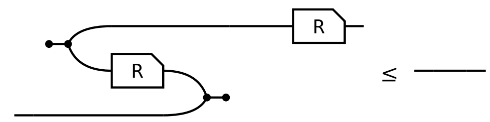
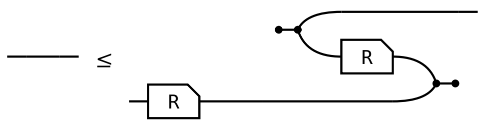
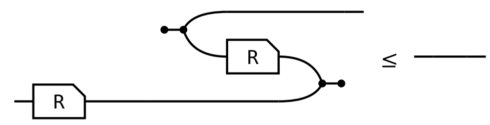
`S ⊸ ⊑ ⊸`, the lax axiom for ⊸ in the first section
— the discard slides back past `S`

Equality is `S` entire, which is the same picture read as `Dom R = 𝟙 ⟺ R` entire.

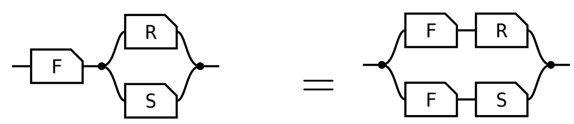
§6. Maps

Every arrow is lax for $\lhd$ and for ⊸ by axiom, and for $\rhd$ and ⟜ by taking $^\circ$ of those two. All four hold for **every** arrow, their REVERSES do not, and each reverse holding is a property of the arrow. The right-hand column is the **adjoint** form, which is the definition used here because it is literally the allegory's `Simple` and `Entire`; that the two forms agree is a separate theorem, not proved here.

(6a)

 **surjective** $\mathbb{1} \sqsubseteq R^\circ\ R$, that is, R° entire.	 **single valued** $R^\circ\ R \sqsubseteq \mathbb{1}$
 **entire** $\mathbb{1} \sqsubseteq R\ R^\circ$. With **single valued**, a **map**.	 **injective** $R\ R^\circ \sqsubseteq \mathbb{1}$

(6b)

law	picture
F (R ∩ S) = F R ∩ F S F single valued F = people x may ask, R = admires a, S = admires b. Single valued means one person, so the sides agree.	 one person who admires both a at A, b at B

§7. / is all of

(7a)

x (R/S) y ⟺ ∀p. y S p → x R p

x (S\R) y ⟺ ∀p. p S x → p R y

/ compares images: S(y) ⊆ R(x). \ compares preimages: S°(x) ⊆ R°(y).

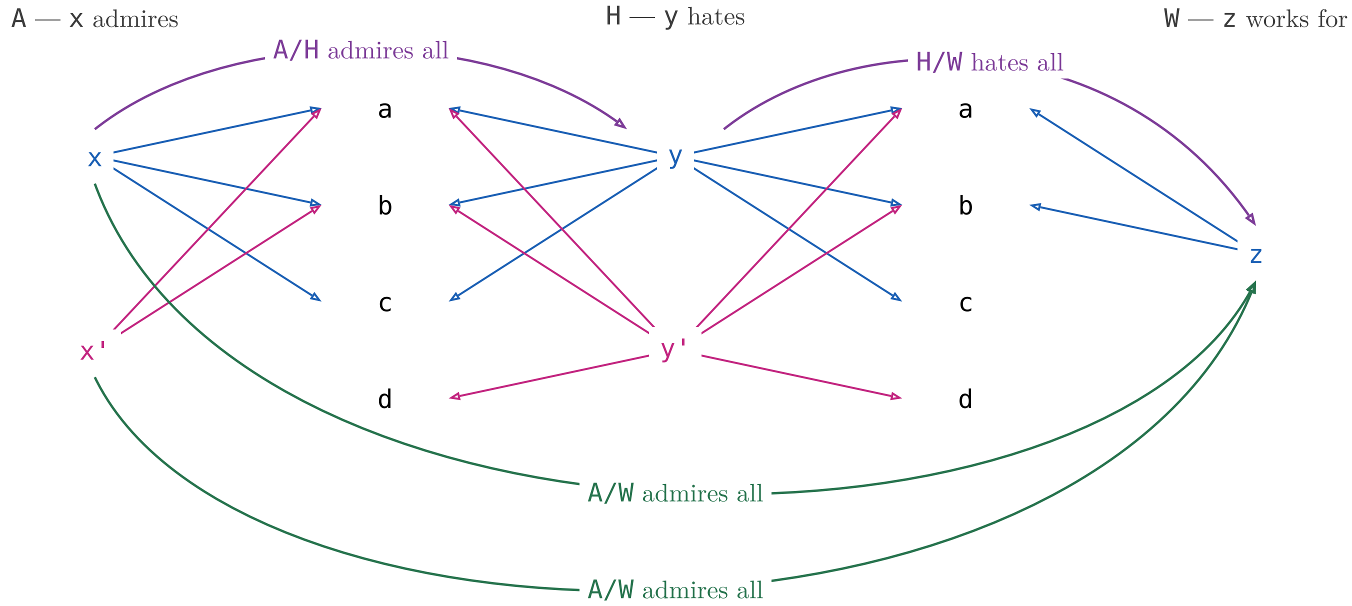
example: A admires, H hates, W works for

x (A/H) y — x admires everyone y hates.

x (H\A) y — everyone who hates x admires y.

§7.1. (R/S)(S/W) ⊑ R/W

(7.1a)



(7.1b)

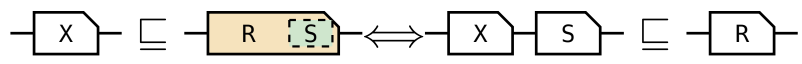
x (A/H) y — x admires everyone y hates
y (H/W) z — y hates everyone z works for
x (A/W) z — x admires everyone z works for

(A/H)(H/W) is a path: x → y → z, and that is all of it. A/W also holds of x', who admires everyone z works for — but x' does not admire everyone anybody hates, so nothing composes to it. The missing path is exactly the strictness of (R/S)(S/W) ⊑ R/W.

(7.1c)

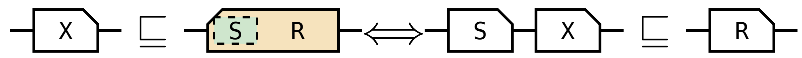
$X \sqsubseteq R/S \iff X\ S \sqsubseteq R$

X is any x-to-y pairing; one that only pairs x with a y such that x admires everyone y hates lies inside R/S, and R/S is the largest such.



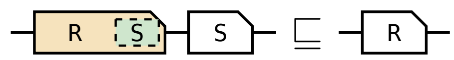
$X \sqsubseteq S\backslash R \iff S\ X \sqsubseteq R$

The mirror — divide on the left when x comes first.



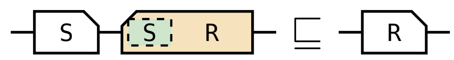
$(R/S)\ S \sqsubseteq R$

There is a y such that x admires everyone y hates, and p is one of the people y hates — then x admires p too. Strict at $S = \varnothing$: R/S is everyone, $(R/S)\ S = \varnothing$.



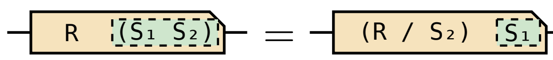
$S\ (S\backslash R) \sqsubseteq R$

The mirror.



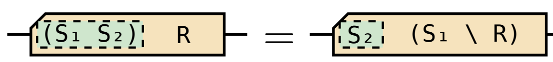
associate: $R/(S_1\ S_2) = (R/S_2)/S_1$

A friend's enemy is two hops: divide by the far end first.



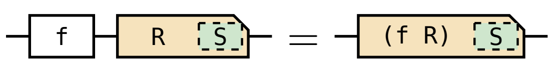
$(S_1\ S_2)\backslash R = S_2\backslash(S_1\backslash R)$

The mirror.



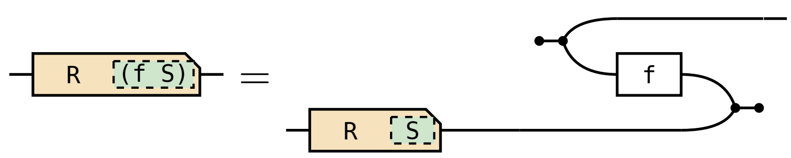
maps: $f\ (R/S) = (f\ R)/S$

Rename x before or after dividing — the licence to write $f\ R\ /\ S$.



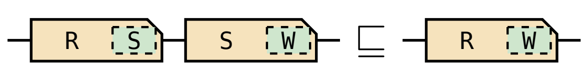
$R/(f\ S) = (R/S)\ f^\circ$

Rename y: a map leaves a denominator as f° outside the box.



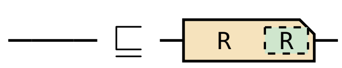
$(R/S)(S/W) \sqsubseteq R/W$

Someone who admires all of a hate-list that already covers everyone z works for admires those people too.



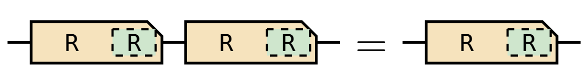
$\mathbb{1} \sqsubseteq R/R$

R/R runs admirer to admirer: each admires everyone they admire. Strict: two people who each admire only a and b admire each other's idols too, and still stay two people.



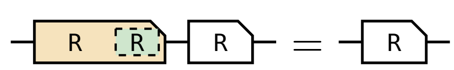
$(R/R)(R/R) = R/R$

R/R is the preorder **admires at least as much as**, and a preorder is idempotent. Freyd writes $\sqsubseteq$; with $\mathbb{1} \sqsubseteq R/R$ above it is an equality.



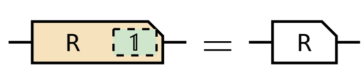
$(R/R)\ R = R$

Reaching p through someone whose idols x fully admires is reaching p directly, since x admires their own idols.



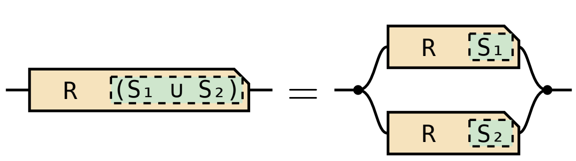
$R/\mathbb{1} = R$

Dividing by $\mathbb{1}$: p's list is just $\{p\}$, so admiring all of it is admiring p.



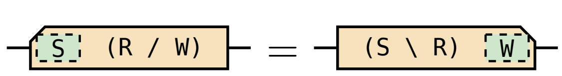
$R/(S_1 \cup S_2) = R/S_1 \cap R/S_2$

Admiring a combined hate-list is admiring each list in full.



$S\backslash(R/W) = (S\backslash R)/W$

Which is why $S\backslash R/W$ needs no bracket.



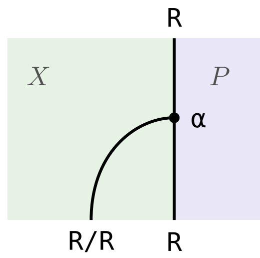
Fifteen laws, fifteen pictures, and not one shows a generator: `∩`, `∪`, `°` and composition are what the Frobenius generators build, and `/` is none of those — it is posited, with nothing to unfold.

§7.2. `R/R` is a preorder, which is what a monad is here

A **monad** in a locally posetal 2-category is a 1-cell `M` with `𝟙 ⊑ M` and `M M ⊑ M` — a unit and a multiplication, and with the hom-posets thin there is nothing else to give. In `Rel` that is a reflexive transitive relation: a preorder. The table above proves both halves at `M := R/R`, and names the preorder they define.

`(R/R) R = R` is then the action that makes `R` a module over that monad — Hinze and Marsden's $\alpha : M \circ A \dot{\to} A$ (IntroString p. 83) at M := `R/R`, A := `R`. Their two coherence conditions are free here: parallel 2-cells are unique, so any two of them are equal.

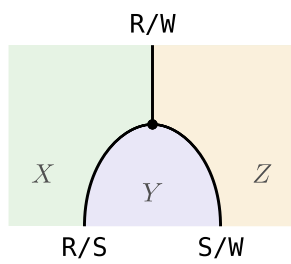
(7.2a)



Green is `X`, where `x` lives; purple is `P`, where the `p` of the definition lives. `R/R : X → X` runs inside `X` and ends on `R : X → P` at the action `α`, which is `(R/R) R ⊑ R`.

§7.3. The whole family is one preorder

(7.3a)



The same dot with one region more: `R/S : X → Y` and `S/W : Y → Z` come in, `R/W : X → Z` goes out. Green `X`, purple `Y`, amber `Z`.

Closing `Y` off is the mid-point `y` being quantified away; the `⊑` rather than `=` is the loss the figure for that law above already draws.

Concretely in `Rel`, `𝟙 ⊑ R/R` and `(R/S)(S/W) ⊑ R/W` are one preorder on the disjoint union of `X`, `Y`, `Z`, …: reflexivity is `𝟙 ⊑ R/R` inside a block, transitivity is `(R/S)(S/W) ⊑ R/W` across blocks.

Abstractly the same two are a **lax functor** out of the codiscrete category on `{R, S, W, …}` — one object per relation, exactly one arrow each way — sending `R` to `X`, the object its rows are indexed by, and the arrow `R → S` to `R/S : X → Y`. That is Bénabou's **polyad** (Bénabou 1967, Def. 5.5), of which a monad is the one-object case: the subsection above is this one at `R = S = W`.

`R/S` is at the same time a bimodule between the two monads `R/R` and `S/S`: `(R/R)(R/S) ⊑ R/S` and `(R/S)(S/S) ⊑ R/S`, each the counit `(R/S) S ⊑ R` with one action composed on — `(R/R) R = R` on the left, `(S/S) S = S` on the right.

§8. `x (Admires%Hates) y` is `x admires only all whom y hates`

(8a)

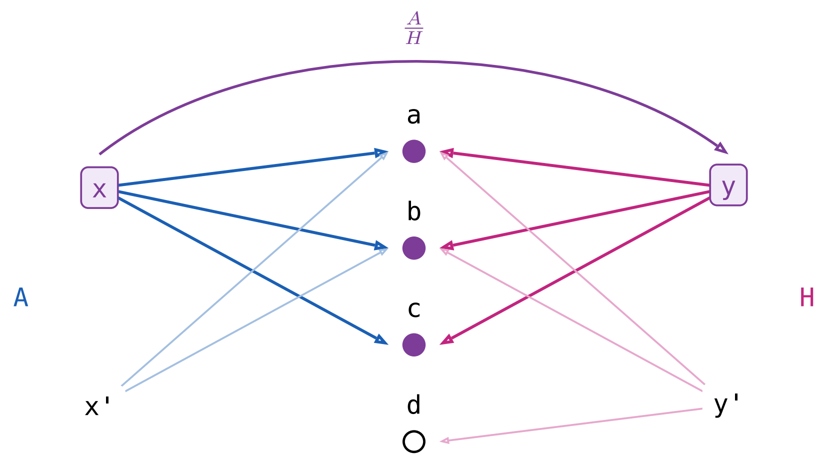
> $\frac{R}{S} \triangleq$ `(R/S) ∩ (S/R)°`. In `Rel` `x` and `y` has the same image: `∀p. (x R p ⟺ y S p)`

(8b)

`x (A/H) y` — `x` admires everyone `y` hates
`x ((H/A)°) y` — `x` admires only people `y` hates
`x ((A/H) ∩ (H/A)°) y` — `x` admires only and all whom `y` hates

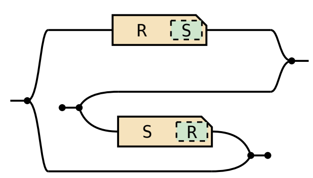
`x` admires exactly whom `y` hates: `/` is **all of**, $\frac{R}{S}$ is **only all of**.

(8c)



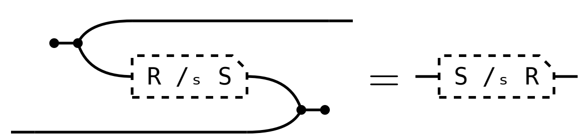
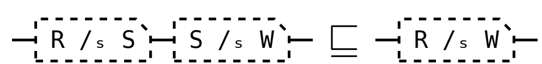
A meet of two long divisions, the second turned round by the converse frame:

(8d)



exported from the definition, not transcribed

(8e)

$\left(\frac{R}{S}\right)^{\circ} = \frac{S}{R}$ Matching is symmetric.	
$\frac{R}{S}\frac{S}{W} \sqsubseteq \frac{R}{W}$ And transitive.	

No pictures for the rest of §2.35: symmetric division is not built from the generators.

(8f)

law	the reading
$X \sqsubseteq \frac{R}{S} \iff XS \sqsubseteq R$ and `X° R ⊑ S`	`X` may pair `x` with `y` only when `x` admires exactly whom `y` hates. Both halves must typecheck, so the operation is **partial**.
$\frac{R}{S}S \sqsubseteq R$	$(\exists y.\, x(\frac{R}{S})y \wedge ySp) \to xRp$ `x only admires whom y hates` $\frac{R}{S}S = \mathrm{Dom}(\frac{R}{S})R$

$\frac{R}{R}R = R$	$(\exists y.\, x(\frac{R}{R})y \wedge yRp) \iff xRp$
	`x and y admire the same people`
	`y = x always qualifies (𝟙 ⊑ R%R below)`

$\mathbb{1} \sqsubseteq \frac{R}{R}$	$x(\frac{R}{R})y$ if `x` and `y` admires the same peoples.
$(\frac{R}{R})^2 = \frac{R}{R}$	So the relation **admires the same people** is an equivalence relation.
$X \sqsubseteq \frac{R}{R} \iff XR \sqsubseteq R$, for symmetric `X`	The largest symmetric arrow that leaves `R` alone.
$\frac{R}{\mathbb{1}}$ is the **simple part** of `R`	The people who admire exactly one person and nobody else. It equals `R` only when `R` is simple, unlike `R/𝟙 = R`.
`Dom` $\frac{R}{S}$ `= 𝟙 ∩ (R/S)(S/R)`	Its domain is the **domain of simplicity** of `R`.

§8.1. Straight

(8.1a)

> `S` is **straight** when $\frac{S}{S} = \mathbb{1}$ — no two `y`s hate the same people.

(8.1b)

law	the reading
every symmetric `X` with `X S ⊑ S` is coreflexive	Equivalently, `S` is straight.
`f S = g S` $\Longrightarrow$ `f = g`	A straight `S` tells its `y`s apart, so it cancels on the right.
`S R` straight $\Longrightarrow$ `S` straight	If the longer chain tells them apart, the first step already does.
`S` straight $\iff$ `R/S` simple for all `R`	If no two `y`s agree, at most one `x` can match a given `y`.
`R = h S`, `h` a cover, `S` straight	In an effective division allegory every arrow factors that way.

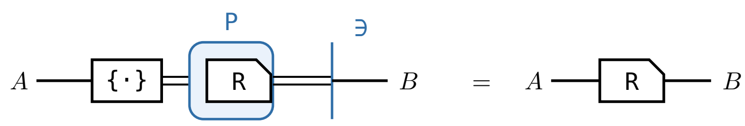
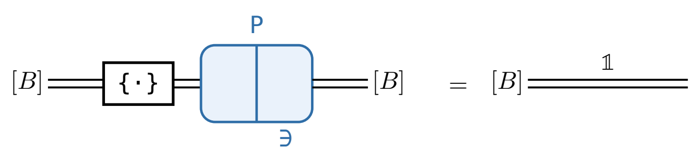
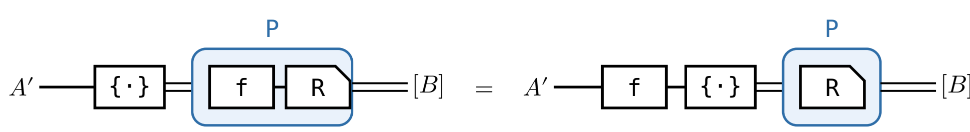
§9. Power allegories

§9.1. ∋ and ∧

A **power allegory** is a division allegory with one unary operation on arrows, ∋ **epsiloff**, subject to (9.1a)

$$
\begin{array}{ll}
\ni_{\mathrm{R}}\Box = R\Box, \quad \ni_{\mathrm{R}} = \ni_{\mathrm{R}\,\Box} & \\
\mathbb{1} \sqsubseteq (R/\ni_{\mathrm{R}})(\ni_{\mathrm{R}}/R) & \ni_{\mathrm{R}} \text{ is } \textbf{thick} \\
(\ni_{\mathrm{R}}/\ni_{\mathrm{R}}) \cap (\ni_{\mathrm{R}}/\ni_{\mathrm{R}})^{\circ} \sqsubseteq \mathbb{1} & \ni_{\mathrm{R}} \text{ is } \textbf{straight}
\end{array}
$$

`R □` is `R`'s target, an identity arrow. For `R : A → B` write `∋ : [B] → B`, dropping the subscript.

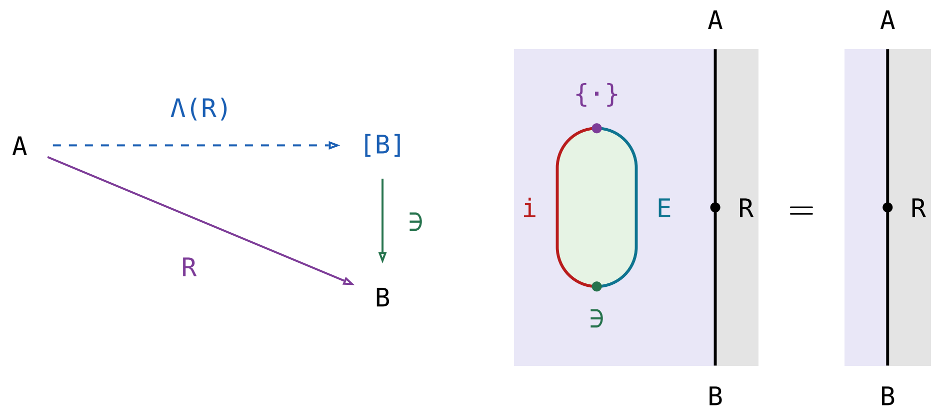
law	the reading
$\Lambda(R) \triangleq \frac{R}{\ni}$: A → [B], for R : A → B	convert a relation to a function
$\ni_R\Box = R\Box,\ \ \ni_R = \ni_{R\,\Box}$	∋ has the same target as R, and replacing R by the identity at that target leaves it unchanged: one ∋ per object, not per arrow.
∋ is **thick**	**Comprehension**: every x has a list of exactly the people x admires. Equivalently every R factors as a map followed by ∋.
∋ is **straight**, that is $\frac{\ni}{\ni} \sqsubseteq 1\!\!1$	**Extensionality**: two lists with the same people are the same list.
Λ(R) is simple	∋ is straight, and dividing by a straight arrow is simple. At most one list per x.
Λ(R) is entire ⟺ ∋ thick	Dom $\frac{R}{\ni}$ = 𝟙 ∩ (R/∋)(∋/R), the domain row above. At least one list per x, so Λ(R) is a **map**.
Λ(R) ∋ = R	
Λ(R) is the only map with Λ(R) ∋ = R	Two maps naming the same people name the same list — extensionality again.
F ⊑ Λ(F ∋), F simple	A partial choice of lists is inside the total one.
[α] = source of ∋, the **power-object**	Every list over α.
{·} ≜ Λ(𝟙), the **singleton map**, monic	The one-person list. Λ(𝟙)Λ°(𝟙) ⊑ $\frac{1\!\!1}{\ni}\frac{\ni}{1\!\!1} \sqsubseteq \frac{1\!\!1}{1\!\!1} \sqsubseteq 1\!\!1$.
Λ(∋) = 𝟙 Make the list of a list, then read it back one level down. Straightness itself, and one of the two triangle identities of i ⊣ P; the other is {·} ∋ = 𝟙.	
fusion: Λ(f R) = f Λ(R), f a map f is a plain rectangle — a map, so it may cross the {·} node. That is naturality of the unit, f {·} = {·} (P f), and it is the whole content of fusion; a chamfered box is stuck on the side it is on.	
Λ(f) = f {·}, f a map	Rename first or take singletons first — the fusion row above at R = 𝟙.
$\frac{R}{S} = \Lambda(R)\Lambda^{\circ}(S)$ x and y match exactly when the two boxes hold one and the same list — the [B]-wire joining them is the whole statement.	
C a topos ⟹ Rel(C) a power allegory	And back: a unitary tabular power allegory has Map(A) a topos. Extensionality and comprehension are all a topos adds.

(9.1b)

§9.2. Λ is the transpose, {·} the unit and ∋ the counit of i ⊣ E

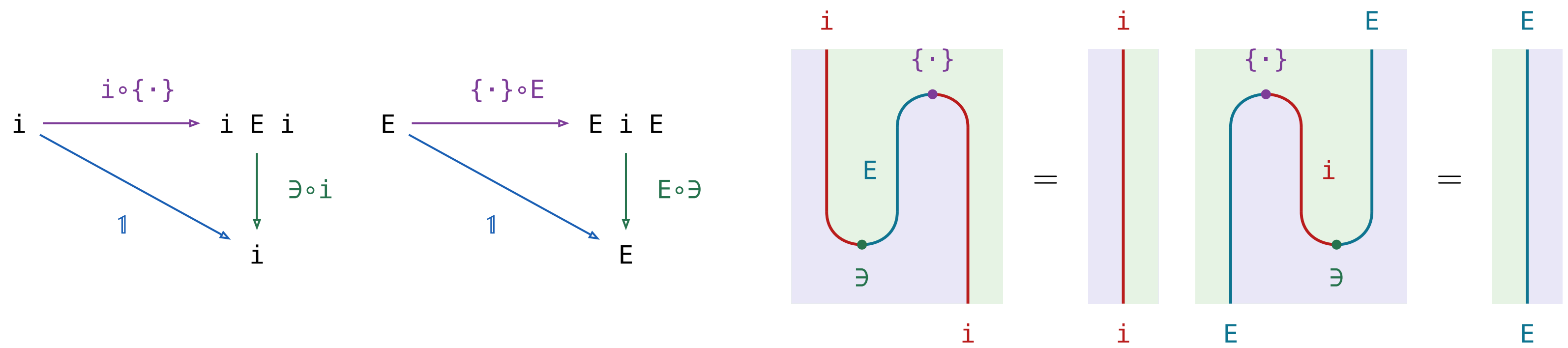
E R ≜ Λ(∋ R) is Bird & de Moor's **existential image** (p. 105), and it is the right adjoint of the inclusion i : Map($\mathcal{A}$) ↪ $\mathcal{A}$: Λ is the transpose, so $\mathcal{A}$(A, B) ≅ Map(A, [B]).

(9.2a)



$\mathtt{\Lambda(R)\ \ni\ =\ R}$, and $\mathtt{\Lambda(R)}$ is the only **map** with that property

(9.2b)



$\mathtt{\{\cdot\}\ \ni\ =\ \mathbb{1}}$ $\qquad$ $\mathtt{\Lambda(\ni)\ =\ \mathbb{1}}$

`E` is not the power relator `P R` of §10.5, for which `(E R) ∋ = ∋ R` weakens to `(P R) ∋ ⊑ ∋ R`.

§10. Relator

(10a)

Every hom-set of an allegory is a poset, so an allegory is a **locally posetal 2-category**: the 2-cell from $\mathtt{R}$ to $\mathtt{S}$ IS $\mathtt{R} \sqsubseteq \mathtt{S}$. A **relator** $\mathtt{F} : \mathcal{C} \to \boldsymbol{\mathcal{D}}$ is a 2-functor between allegories:

$$\mathtt{F}\ \mathbb{1} = \mathbb{1} \qquad \mathtt{F(R\ S) = (F\ R)(F\ S)} \qquad \mathtt{R} \sqsubseteq \mathtt{S} \implies \mathtt{F\ R} \sqsubseteq \mathtt{F\ S}$$

Preserving $^\circ$ is **not** asked for — $^\circ$ is an identity-on-objects involution $\mathcal{C}^{op} \to \mathcal{C}$, no part of the 2-category.

- For $\mathtt{f}$ a map, $\mathtt{F\ f}$ is a map and $\mathtt{F(f^\circ) = (F\ f)^\circ}$.
- Over a **tabular** allegory a functor is a relator $\iff$ it preserves $^\circ$.
- A relator is fixed by what it does to maps.
- $\mathtt{F(R \cap S)} \sqsubseteq \mathtt{(F\ R) \cap (F\ S)}$, and strictly.
- $\mathtt{F(X \cap Y) = (F\ X) \cap (F\ Y)}$ for $\mathtt{X,\ Y}$ coreflexive.
- $\mathtt{F(Dom\ R) = Dom(F\ R)}$ for $\mathtt{F}$ preserving $^\circ$.

The **power relator** $\mathtt{P}$ — $\mathtt{x\ (P\ R)\ y} \iff \mathtt{(\forall a \in x.\ \exists b \in y.\ a\ R\ b) \wedge (\forall b \in y.\ \exists a \in x.\ a\ R\ b)}$ — is where the fourth is strict: for $\mathtt{R = \{(a_1,b_1),\ (a_2,b_2)\}}$ and $\mathtt{S = \{(a_1,b_2),\ (a_2,b_1)\}}$ the pair $\mathtt{(\{a_1,a_2\},\ \{b_1,b_2\})}$ is in $\mathtt{P\ R \cap P\ S}$, while $\mathtt{R \cap S = \varnothing}$.

§10.1. The fork $\langle \mathtt{R,S} \rangle$

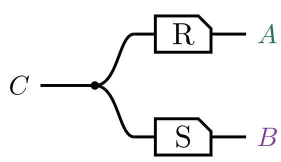
(10.1a)

The **fork** of $\mathtt{R : C \to A}$ and $\mathtt{S : C \to B}$ is $\mathtt{\langle R,S \rangle \triangleq R\ \pi_1^\circ \cap S\ \pi_2^\circ}$, where $(\pi_1,\ \pi_2)$ is the tabulation of $\top$.

$$\mathtt{\langle R,S \rangle\ \pi_1 = (Dom\ S)\ R} \qquad \mathtt{\langle R,S \rangle\ \pi_2 = (Dom\ R)\ S} \tag{10.1b}$$

(10.1c)

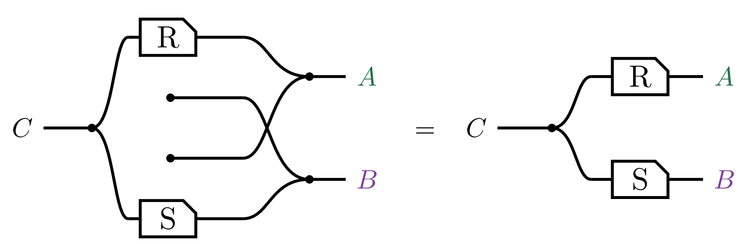
$$\begin{tikzcd}
 & A & \\
C \arrow[ur, "\mathtt{R}"] \arrow[dr, "\mathtt{S}"'] \arrow[rr, dashed, "\langle R,S \rangle"] & \sqsupseteq \quad \sqsupseteq & A \times B \arrow[ul, "\pi_1"'] \arrow[dl, "\pi_2"] \\
 & B &
\end{tikzcd}$$



A domain is coreflexive, so $\mathtt{\langle R,S \rangle\ \pi_1} \sqsubseteq \mathtt{R}$, with equality exactly when $\mathtt{S}$ is entire; for maps both triangles commute and $\mathtt{\langle f,g \rangle}$ is unique. In $\mathtt{Rel}$, $\mathtt{c\ \langle R,S \rangle\ (a,b)}$ iff $\mathtt{c\ R\ a}$ and $\mathtt{c\ S\ b}$ — copy $\mathtt{c}$, then $\mathtt{R}$ on one strand and $\mathtt{S}$ on the other, which is $\mathtt{\lhd\ (R \otimes S)}$ on the right.

No $^\circ$ survives the translation. $\pi_1 = \mathbb{1} \otimes \multimap$ discards the second component, so $\pi_1^\circ = \mathbb{1} \otimes \multimapinv$ **creates** one out of nothing, and $\cap$ is copy, run both, merge. Draw that and the created strands — the two dots with no left end, and the crossing they force — are merged against real ones, which is the monoid's unit law:

(10.1d)



$\multimapinv\ \rhd\ =\ \mathbb{1}$ on each half

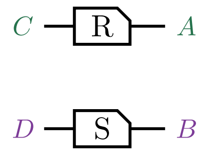
§10.2. The relational product $\mathtt{R \times S}$

(10.2a)

$\mathtt{R \times S \triangleq \langle \pi_1\ R,\ \pi_2\ S \rangle}$, a relator in each argument but no longer a categorical product.

(10.2b)

$$\begin{tikzcd}
C \arrow[r, "R"] \arrow[dr, phantom, "\sqsupseteq"] & A \\
C \times D \arrow[u, "\pi_1"] \arrow[r, dashed, "R \times S"] \arrow[d, "\pi_2"'] \arrow[dr, phantom, "\sqsupseteq"] & A \times B \arrow[u, "\pi_1"'] \arrow[d, "\pi_2"] \\
D \arrow[r, "S"] & B
\end{tikzcd}$$

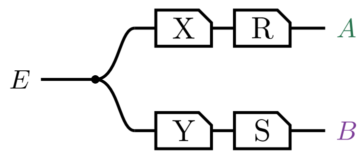


Right-then-up is $(R \times S)\ \pi_1$, up-then-right is $\pi_1\ R$, and $(R \times S)\ \pi_1 \sqsubseteq \pi_1\ R$, equality when S is entire. In Rel, $(c,d)\ (R \times S)\ (a,b)$ iff $c\ R\ a$ and $d\ S\ b$ — two strands side by side, no copy dot: $R \times S = R \otimes S$.

§10.3. Absorption

For $X : E \to C$ and $Y : E \to D$, $\langle X,Y\rangle\ (R \times S) = \langle X\ R,\ Y\ S\rangle$. Both sides are this picture:

(10.3a)



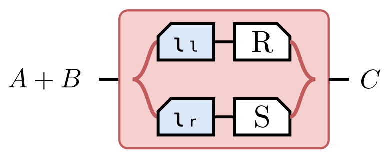
§10.4. The coproduct [R,S]

The injections $\iota_l : A \to A + B$ and $\iota_r : B \to A + B$ are maps, and the coproduct they make of the maps stays a coproduct once every arrow is allowed: both equations hold with equality and $[R,S]$ is the only arrow satisfying them, with none of the Dom slack $\langle R,S\rangle$ carries.

(10.4a)

> $[R,S] \triangleq \iota_l{}^\circ\ R \cup \iota_r{}^\circ\ S$, and $R + S \triangleq [R\ \iota_l,\ S\ \iota_r]$.

(10.4b)



The tape is the union — a particle entering at $A + B$ takes exactly one branch — and the two mirrored boxes are what makes the branches disjoint.

(10.4c)

$\iota_l\ [R,S] = R$, $\iota_r\ [R,S] = S$, and $[R,S]$ is the only such arrow
$\iota_l\ \iota_l{}^\circ = \mathbb{1} = \iota_r\ \iota_r{}^\circ$
$\iota_l\ \iota_r{}^\circ = \bot = \iota_r\ \iota_l{}^\circ$
$\iota_l{}^\circ\ \iota_l \cup \iota_r{}^\circ\ \iota_r = \mathbb{1}$
$[U,V]^\circ\ [R,S] = U^\circ\ R \cup V^\circ\ S$

The first row is not free: $\iota_l,\ \iota_r$ were only ever asked to be a coproduct of **maps**. They stay one once every arrow is allowed because Λ sends an arrow $A \to C$ to a map $A \to [C]$ reversibly, so the map coproduct can be applied underneath it. For any $T : A + B \to C$,

(10.4d)

$$\begin{aligned}
& \iota_l\ T = R \text{ and } \iota_r\ T = S \\
\iff\ & \quad \text{Λ is injective — $\Lambda(X)\ \ni\ = X$ gives X back} \\
& \Lambda(\iota_l\ T) = \Lambda(R) \text{ and } \Lambda(\iota_r\ T) = \Lambda(S) \\
\iff\ & \quad \text{Λ fusion, $\Lambda(f\ X)\ = f\ \Lambda(X)$ for f a map} \\
& \iota_l\ \Lambda(T) = \Lambda(R) \text{ and } \iota_r\ \Lambda(T) = \Lambda(S) \\
\iff\ & \quad \text{the coproduct of maps, at the map $\Lambda(T)$} \\
& \Lambda(T) = [\Lambda(R),\ \Lambda(S)] \\
\iff\ & \quad \text{$\Lambda(X)$ is the only map with $\Lambda(X)\ \ni\ = X$} \\
& T = [\Lambda(R),\ \Lambda(S)]\ \ni
\end{aligned}$$

Every step is an $\Longleftrightarrow$, so $[\Lambda(R),\ \Lambda(S)]\ \ni$ is the **only** arrow satisfying the two equations — which is what $[R,S]$ was claimed to be. Drawn:

$$
\begin{tikzcd}
 & A \arrow[dl, "\iota_l"'] \arrow[dr, "\Lambda(R)"] \arrow[drr, bend left, "R"] & & \\
A+B \arrow[rr, dashed, "{[\Lambda(R),\ \Lambda(S)]}"] & & {[C]} \arrow[r, "\ni"] & C \\
 & B \arrow[ul, "\iota_r"] \arrow[ur, "\Lambda(S)"'] \arrow[urr, bend right, "S"'] & &
\end{tikzcd}
\tag{10.4e}
$$

Nothing here holds only up to $\sqsubseteq$: every triangle commutes on the nose, which is the difference from the fork above. The border spells $[R,S] = [\Lambda(R),\ \Lambda(S)]\ \ni$, and pushing $\ni$ into the union that $[\cdot,\cdot]$ on maps already is turns that back into the definition, $(\iota_l^\circ\ \Lambda(R)\ \cup\ \iota_r^\circ\ \Lambda(S))\ \ni\ =\ \iota_l^\circ\ R\ \cup\ \iota_r^\circ\ S$.

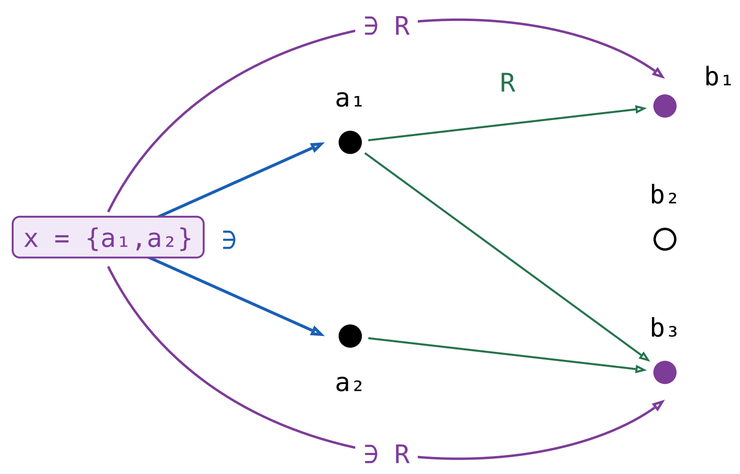
§10.5. The power relator `P R`

For `R : A → B`, `P R ≜ ((∋ R)/∋) ∩ ((∋ R°)/∋)° : [A] → [B]`. (10.5a)

`∋ R` is a **composition** — `∋ : [A] → A` followed by `R` — and not Freyd's subscripted $\ni_R$, which is the `∋` at `R`'s **target**: §9 writes one `∋` per object and drops the subscript. Piece by piece, on one instance, with `x, y` lists and `a, b` their elements.

`x = {a₁,a₂}`, and `R` sends `a₁` to `b₁` and `b₃`, `a₂` to `b₃` alone. Nothing on `x` reaches `b₂`, so `b₂` is drawn hollow.

(10.5b)

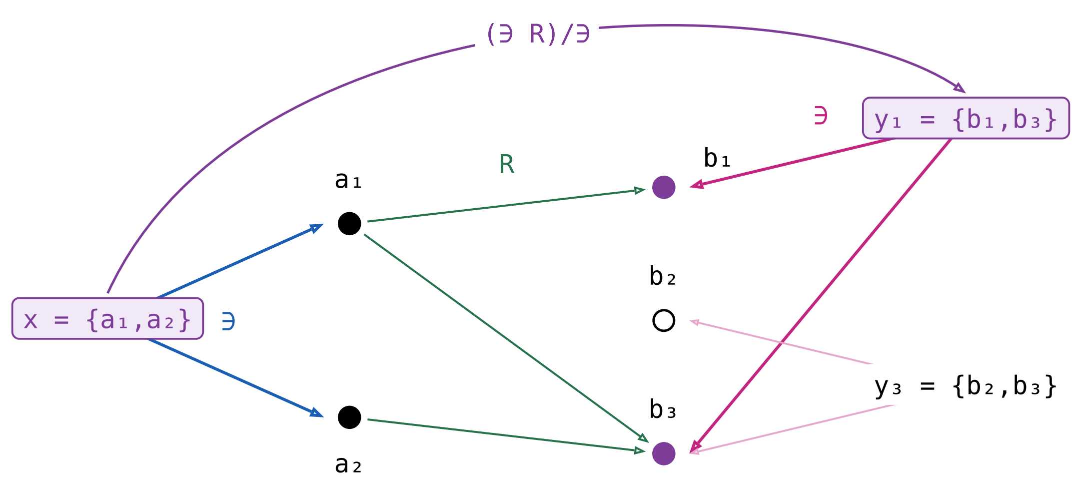


`∋ R : [A] → B`   `x (∋ R) b ⟺ ∃a ∈ x. a R b`

one arc per element of `B` that `x` reaches, and `b₂` gets none

Dividing by `∋` turns that into a relation between **lists**: every element of `y` must be one of the filled dots.

(10.5c)

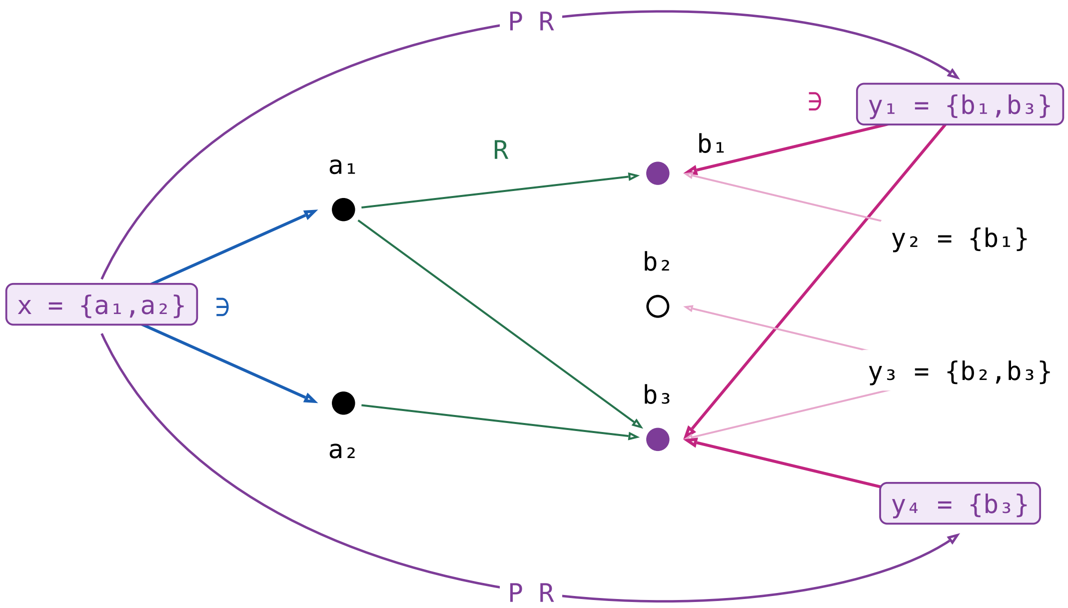


`(∋ R)/∋ : [A] → [B]`   `x ((∋ R)/∋) y ⟺ ∀b ∈ y. ∃a ∈ x. a R b`

`y₃` is rejected: it names `b₂`, which `x` does not reach

The other half of the meet is that clause with the lists swapped and R turned round: every element of x must reach y. Both together, and two more lists to separate them:

(10.5d)



$$((\ni\ R^\circ)/\ni)^\circ : [A] \to [B] \qquad x\ ((\ni\ R^\circ)/\ni)^\circ\ y \iff \forall a \in x.\ \exists b \in y.\ a\ R\ b$$

y_2 is rejected too: a_2's only image is b_3, which y_2 does not name

In words, if $x\ (P\ R)\ y$ then every element of x is related by R to some element of y, and conversely — the two clauses of $\cap$, one per direction.

Neither $\ni$ nor $P\ R$ is a map. $\ni$ sends a list to **each** of its elements; $P\ R$ sends x to **every** y meeting the two clauses, y_1 and y_4 both. Nor is it entire: an $a \in x$ with no image at all leaves x with no partner whatever. Turning a relation into a map is Λ's job (§9), and on lists that map is $\Lambda(\ni\ R)$ — of the four it accepts y_1 only.

Not a symmetric division. $\frac{\ni R}{\ni}$ is $\Lambda(\ni\ R)$, and in $\mathtt{Rel}$ it reads $x\ \Lambda(\ni\ R)\ y \iff y = \{b \mid \exists a \in x.\ a\ R\ b\}$: in words, y is **exactly** what x reaches, where $P\ R$ asks only that each side cover the other. Nor can it be repaired: in that fraction's second half $(\ni /(\ni R))^\circ$ the R sits in a denominator, so the operation is antitone there, and a relator has to be monotone. $P\ R$ keeps the first half and takes the second half at R°, which puts R back in a numerator. The fraction returns exactly where the two halves agree — at $\mathbb{1}$, and at a map.

(10.5e)

law	**the reading**
$X \sqsubseteq P\ R \iff X\ \ni\ \sqsubseteq\ \ni\ R$ and $X^\circ\ \ni\ \sqsubseteq\ \ni\ R^\circ$	One containment, and the same one at R° — which is the definition read off the two divisions. Hence $P(R^\circ) = (P\ R)^\circ$, and $R \sqsubseteq S \implies P\ R \sqsubseteq P\ S$.
$P\ \mathbb{1} = \frac{\ni}{\ni} = \mathbb{1}$	The straightness axiom verbatim: extensionality **is** P's unit law.
$P\ f = \Lambda(\ni\ f) = \frac{\ni f}{\ni}$, for f a map	In $\mathtt{Rel}$, $x\ (P\ f)\ y \iff y = \{f\ a \mid a \in x\}$. The half at f° says every $a \in x$ has its $f\ a$ on y; f has just the one image per a, so that already says y contains everything x reaches, which is the fraction's second half. For a map the two definitions coincide.
$P(R\ S) = (P\ R)(P\ S)$	$\sqsupseteq$ is the division cancellation laws. $\sqsubseteq$ is the one law in this section that is not a calculation: it needs a tabulation of $P(R\ S)$.

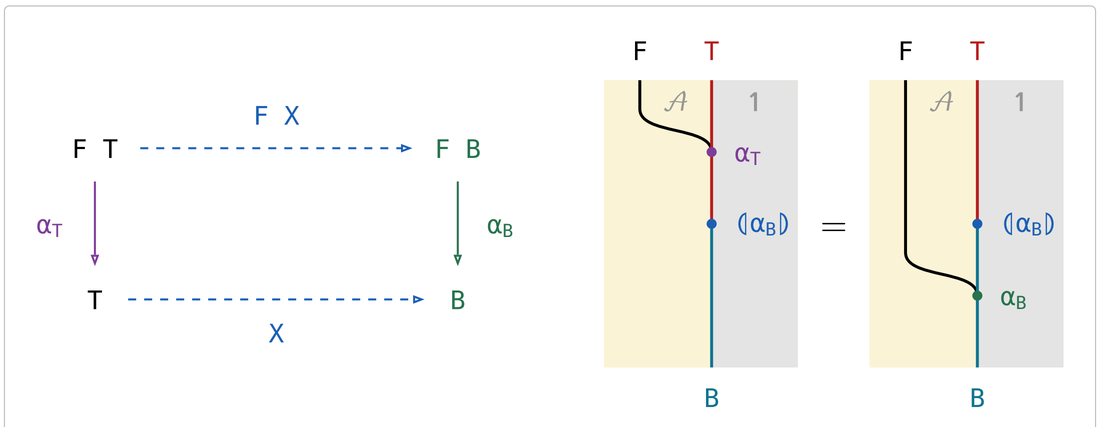
§11. Catamorphism

> F a relator with an **initial algebra** $\alpha_T : F\ T \to T$ among the maps. For a relational algebra $\alpha_B : F\ B \to B$, the **catamorphism** $(\!|\alpha_B|\!) : T \to B$ is the unique arrow with $\alpha_T\ (\!|\alpha_B|\!) = (F\ (\!|\alpha_B|\!))\ \alpha_B$. (11a)

An F-algebra is a **weakened** natural transformation. A transformation $F \Rightarrow \mathbb{1}$ on $\mathcal{A}$ would need a component $F\ X \to X$ at every object and a commuting square at every arrow, but F-Algebra only need it works on T and B.

§11.1. The defining equation

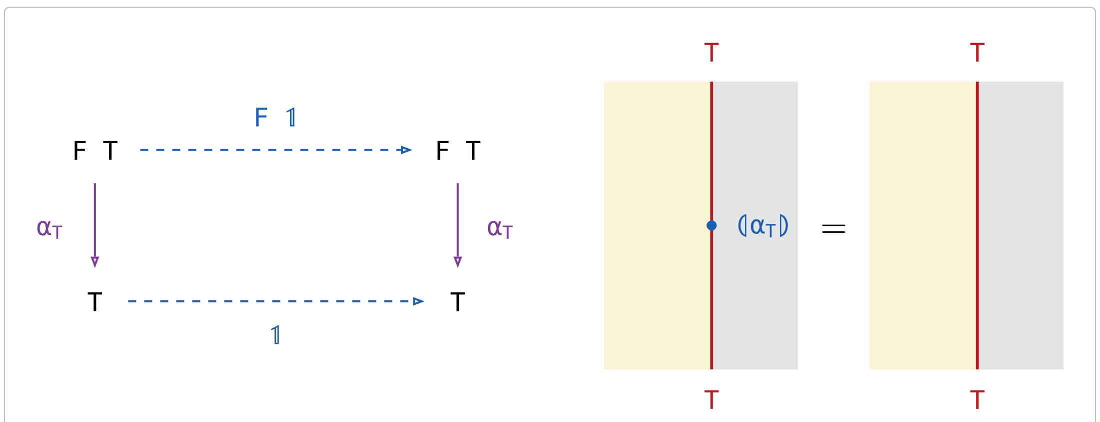
(11.1a)



$$X = (\!|\alpha_B|\!) \iff \alpha_T\ X = (F\ X)\ \alpha_B$$

§11.2. Reflection

(11.2a)



$$(\!|\alpha_T|\!) = \mathbb{1}$$

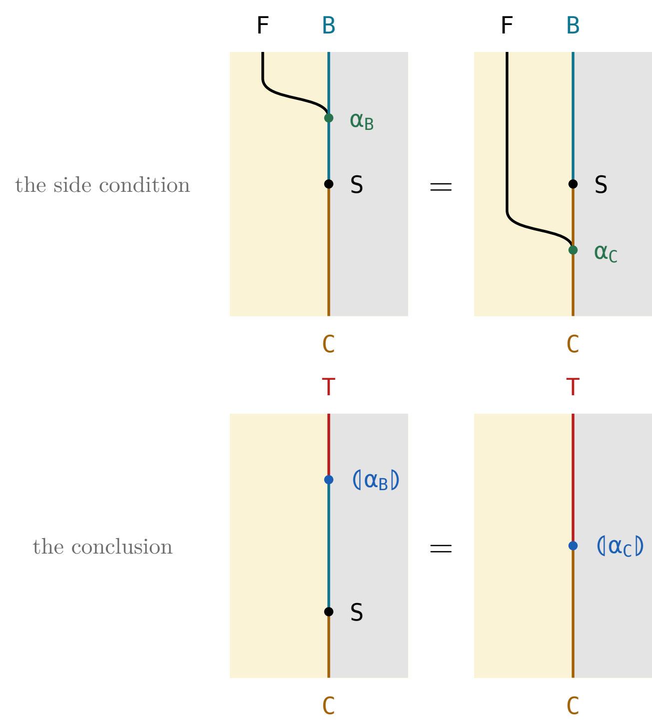
Taking a value apart with α_T and putting it straight back is doing nothing.

§11.3. Fusion

Fusion rewrites $(\!|\alpha_B|\!)\ S$ through a second algebra $\alpha_C : F\ C \to C$ along an arrow $S : B \to C$.

(11.3a)

$$\begin{tikzcd}
F\,T \arrow[r, dashed, "F\,\llparenthesis\alpha_B\rrparenthesis"] \arrow[d, "\alpha_T"'] & F\,B \arrow[r, "F\,S"] \arrow[d, "\alpha_B"] & F\,C \arrow[d, "\alpha_C"] \\
T \arrow[r, dashed, "\llparenthesis\alpha_B\rrparenthesis"'] & B \arrow[r, "S"'] & C
\end{tikzcd}$$



$$\llparenthesis\alpha_B\rrparenthesis\ S = \llparenthesis\alpha_C\rrparenthesis \Longleftarrow \alpha_B\ S = (F\ S)\ \alpha_C$$

The left square is $\llparenthesis\alpha_B\rrparenthesis$'s own defining square and the right one is the side condition, so the outer rectangle says $\llparenthesis\alpha_B\rrparenthesis\ S$ satisfies the defining equation of $\llparenthesis\alpha_C\rrparenthesis$ — and uniqueness finishes it. A fold followed by `S` has collapsed into a single fold, which is how an intermediate structure is got rid of.

The side condition is (11.1a)'s picture again under $\alpha_T \mapsto \alpha_B$, $\llparenthesis\alpha_B\rrparenthesis \mapsto S$, $\alpha_B \mapsto \alpha_C$: both say that the bead between the two merges is a homomorphism of `F`-algebras.

§12. Catamorphism in $\mathcal{S}et$

(12a)

name	definition	note
`listr`	`listr A ::= nil | cons (A, listr A)`	The cons-lists over `A`, the datatype every row below folds (B&dM p. 55).
`sum`	`⦇[zero, plus]⦈`	`plus(a, b) = a + b`.
`length`	`⦇[zero, outr succ]⦈`	`outr` drops the head and keeps the count of the tail, `succ` adds one for the head.
`average`	`⟨sum, length⟩ div`	`div(m, n) = m/n`, with `div(0, 0) = 0` so `average` is total. Traverses the list twice.
banana-split law	`⟨⦇h⦈, ⦇k⦈⟩ = ⦇⟨(F outl) h, (F outr) k⟩⦈`	Any fork of folds is a single fold, hence one traversal — `F` the base functor.
what it reduces to	`α ⟨⦇h⦈, ⦇k⦈⟩ = F⟨⦇h⦈, ⦇k⦈⟩ ⟨(F outl) h, (F outr) k⟩`	All that (11.1a) leaves to check: the fork satisfies the defining equation, and uniqueness finishes it.
the instance	`⟨sum, length⟩ = ⦇[zeros, pluss]⦈`	`zeros = ⟨zero, zero⟩` and `pluss(a, (b, n)) = (a + b, n + 1)`, so `average` runs in one pass.
`preds`	`preds : listr Nat ← Nat`, `preds n = [n, n−1, …, 1]`	B&dM Ex 3.6 (p. 57), posed and not answered there: apply Ex 3.4 to write `preds` as `⦇k⦈ outl`.

The other laws.

The two $\sqsubseteq$ **fusion** rows rewrite $\llparenthesis \alpha_B \rrparenthesis\ S$ through the same α_C and S. They are the only rows here that need the allegory **locally complete** — every hom-set a complete lattice — because they come from a least-fixed-point argument; the rest, the equality fusion (11.3a) included, needs only the initial algebra and the defining equation.

(12b)

name	**law**	**what it says**
Lambek	$\alpha_T{}^\circ = \alpha_T{}^{-1}$, the initial algebra is an isomorphism	A constructor can be undone — $\alpha_T{}^\circ$ takes a value apart into the parts it was built from.
Eilenberg–Wright	$\Lambda \llparenthesis \alpha_B \rrparenthesis = \llparenthesis \Lambda((F\ \ni)\ \alpha_B) \rrparenthesis$	A relational fold is a deterministic fold of SETS: Λ pushes the nondeterminism into the power-object, where the fold is a map again.
Eilenberg–Wright	$\llparenthesis \alpha_B \rrparenthesis = \llparenthesis \Lambda((F\ \ni)\ \alpha_B) \rrparenthesis \ni$	The same fact read back — fold deterministically into a set, then take a member of it.
fusion	$\llparenthesis \alpha_C \rrparenthesis \sqsubseteq \llparenthesis \alpha_B \rrparenthesis\ S \Longleftarrow (F\ S)\ \alpha_C \sqsubseteq \alpha_B\ S$	Half of fusion: an inclusion between the two algebras is inherited by the folds.
fusion	$\llparenthesis \alpha_B \rrparenthesis\ S \sqsubseteq \llparenthesis \alpha_C \rrparenthesis \Longleftarrow \alpha_B\ S \sqsubseteq (F\ S)\ \alpha_C$	The other half, with both inclusions turned around.

The two Λ rows, drawn:

(12c)

$$\begin{tikzcd}
F\ T \arrow[r, dashed, "F\ K"] \arrow[d, "\alpha_T"'] & F\ [B] \arrow[r, "F\ \ni"] \arrow[d, "\Lambda((F\ \ni)\ \alpha_B)"] & F\ B \arrow[d, "\alpha_B"] \\
T \arrow[r, dashed, "K"'] & {[B]} \arrow[r, "\ni"'] & B
\end{tikzcd}$$

The left square is that catamorphism's own defining square, $K = \llparenthesis \Lambda((F\ \ni)\ \alpha_B) \rrparenthesis$, and the right one is Λ's cancellation, so the outer rectangle says $K \ni$ satisfies the defining equation of $\llparenthesis \alpha_B \rrparenthesis$ — and uniqueness finishes it.

§12.1. Fokkinga's mutual recursion theorem

- `F-Alg(𝓐)` — the algebras and their homomorphisms — has binary products whenever $\mathcal{A}$ does, and the forgetful `U : F-Alg(𝓐) → 𝓐` **creates** them: the product of `h : F A → A` and `k : F B → B` is carried by `A × B`, with structure `⟨(F outl) h, (F outr) k⟩ : F(A × B) → A × B`.
- `outl` and `outr` are homomorphisms out of it, and `⟨p, q⟩ : X → A × B` is a homomorphism **iff** `p` and `q` both are — substitute the structure and compare the two forks component by component.
- The banana-split law of (12a) IS that product's universal property read at the initial algebra: `⦇h⦈` and `⦇k⦈` are the unique homomorphisms to `(A, h)` and `(B, k)`, so their fork is the unique homomorphism into the product, which is `⦇⟨(F outl) h, (F outr) k⟩⦈`.
- Fokkinga's theorem is **strictly more general** and is not that product: in (12.1a) the two arrows leave `F(A × B)`, not `F A` and `F B`, so each sees BOTH components. The product is the case where they factor as `(F outl) h` and `(F outr) k`; B&dM's Ex 3.4 is the other special case (p. 58). What it says: **any algebra on a product carrier is folded by a fork.**

$$
\begin{tikzcd}
F\,T \arrow[r, dashed, "F\langle f,\, g\rangle"] \arrow[d, "\alpha"'] & F(A \times B) \arrow[d, "h"] \\
T \arrow[r, dashed, "f"'] & A
\end{tikzcd}
\qquad
\begin{tikzcd}
F\,T \arrow[r, dashed, "F\langle f,\, g\rangle"] \arrow[d, "\alpha"'] & F(A \times B) \arrow[d, "k"] \\
T \arrow[r, dashed, "g"'] & B
\end{tikzcd}
$$

$$\alpha\ f = F\langle f,\, g\rangle\ h \ \wedge\ \alpha\ g = F\langle f,\, g\rangle\ k \ \equiv\ \langle f,\, g\rangle = \llparenthesis \langle h,\, k\rangle \rrparenthesis \tag{12.1a}$$

§12.2. Ruby triangles

(12.2a)

stage	definition
informally (p. 58)	$\mathtt{tri\ f\ [a_0,\ a_1,\ \dots,\ a_i,\ \dots,\ a_n] = [a_0,\ f\ a_1,\ \dots,\ f^i\ a_i,\ \dots,\ f^n\ a_n]}$
for cons-lists (p. 59)	`tri f = ⦇[nil, (𝟙 × listr f) cons]⦈`
with the base functor named	`F(A, B) = 1 + A × B`, `α = [nil, cons]`, so `tri f = ⦇F(𝟙, listr f) α⦈`
abstractly (p. 59)	`F` a bifunctor with initial type `(α, T)`: `tri f = ⦇F(𝟙, T f) α⦈`

For the definition to make sense `f : A → A` is required, and then `tri f : T A → T A`.

$$
\begin{tikzcd}
A \arrow[d, "f"'] \\
A
\end{tikzcd}
\qquad
\begin{tikzcd}
F(A,\, A) \arrow[d, "g"'] \\
A
\end{tikzcd}
\qquad
\begin{tikzcd}
T\,A \arrow[rr, "\mathrm{tri}\ f"] \arrow[dr, dashed, "\llparenthesis F(\mathbb{1},\, f)\ g \rrparenthesis"'] & & T\,A \arrow[dl, dashed, "\llparenthesis g \rrparenthesis"] \\
& A &
\end{tikzcd}
$$

$$(\mathrm{tri}\ f)\ \llparenthesis g \rrparenthesis = \llparenthesis F(\mathbb{1},\, f)\ g \rrparenthesis \quad \Longleftarrow \quad g\ f = F(f,\, f)\ g \tag{12.2b}$$

(12.2b) is Horner's rule, generalised from cons-lists to any initial type; B&dM call it that because for cons-lists it is the schoolbook method for evaluating a polynomial (p. 58). Fusion reduces it to `F(𝟙, T f) α ⦇g⦈ = F(𝟙, ⦇g⦈) F(𝟙, f) g` (p. 59).

§12.3. Depth of a tree

(12.3a)

the datatype	`tree A ::= tip A | bin (tree A, tree A)`, base functor `F(A, B) = A + B × B`, initial type `([tip, bin], tree)`
the fold	`⦇[g, h]⦈` is the unique `f` with `f (tip a) = g a` and `f (bin (x, y)) = h (f x, f y)`
`tree f`	`tree f = ⦇F(f, 𝟙) [tip, bin]⦈`; pointwise `tree f (tip a) = tip (f a)` and `tree f (bin (x, y)) = bin (tree f x, tree f y)`
`max`	`max = ⦇[𝟙, bmax]⦈`, where `bmax (a, b)` is the larger of `a` and `b`
`depths`	`depths = (tree zero) (tri succ)` — replaces every tip by its depth in the tree, `zero` the constant function returning 0 and `succ` the successor
`depth`	`depth = depths max`

§13. Type functor

(13a)

F a **binary** relator: $F(R,\ S)$ is its action on a pair, and $F\ X$ abbreviates $F(\mathbb{1},\ X)$, the F of the catamorphism section. For every object A the initial algebra is $\alpha : F(A,\ T\ A) \to T\ A$, among the maps. The **type functor** T acts on an arrow $R : A \to B$ by

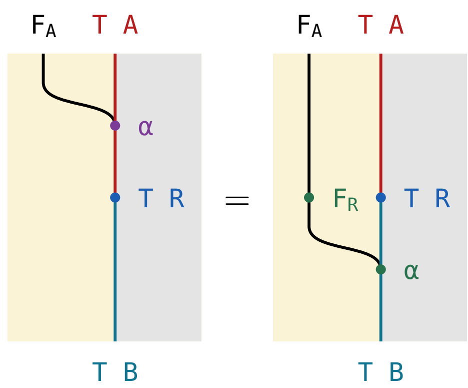
$$T\ R = (\!|F(R,\ \mathbb{1})\ \alpha|\!) : T\ A \to T\ B$$

F is the **base functor** — one layer of the structure, acting on the recursive position — while T is the datatype itself, acting on the parameter. For cons-lists, `list R = ([nil, (R ⊗ 𝟙) cons])`, which is `map R`.

A bifunctor puts its two arguments in two different places. The wire is the partial application $F_A = F(A,\ -)$, and an arrow in the SECOND argument is a bead on the object wire with the F wire running past it — the catamorphism section's rule, unchanged — while an arrow $R : A \to B$ in the FIRST induces a natural transformation $F_R : F_A \Rightarrow F_B$, which is a bead ON the F wire. Marsden draws bifunctors exactly so (`CatString.pdf`, **Bifunctors**): side-by-side is already spent on functor composition, so one argument has to go into the wire's name.

(13b)

$$\begin{tikzcd}
F(A,\ T\ A) \arrow[r, dashed, "F(\mathbb{1},\ T\ R)"] \arrow[d, "\alpha"'] & F(A,\ T\ B) \arrow[d, "F(R,\ \mathbb{1})\ \alpha"] \\
T\ A \arrow[r, dashed, "T\ R"'] & T\ B
\end{tikzcd}$$



$$T\ R = (\!|F(R,\ \mathbb{1})\ \alpha|\!)$$

$T\ R$ is the unique arrow making it commute; there is no $\sqsubseteq$ in it.

Two beads at one height on two different wires are one action, here $F(R,\ T\ R)$, so the right panel reads $F(R,\ T\ R)\ \alpha$ and the left one $\alpha\ (T\ R)$. Their relative height says nothing: slide F_R below the $T\ R$ bead and the panel reads $F(\mathbb{1},\ T\ R)\ F(R,\ \mathbb{1})$ and then α, which is the square's top row followed by its right vertical; slide it above and the panel reads $F(R,\ \mathbb{1})\ F(\mathbb{1},\ T\ R)$, the route through $F(B,\ T\ A)$ the square leaves out. Identifying those two is the exchange condition every bifunctor satisfies (Hinze & Marsden, Ex 1.23), and in this calculus there is nothing to identify.

(13c)

name	law	what it says
the defining equation	$T\ R = (\!|F(R,\ \mathbb{1})\ \alpha|\!)$	Rebuild the structure with α, applying R to the parameter on the way; that is `map R`.
functor	$T\ \mathbb{1} = \mathbb{1}$ and $(T\ R)(T\ S) = T\ (R\ S)$	Mapping the identity changes nothing, and two maps in a row are one map.
type functor fusion	$(T\ R)\ (\!|Q|\!) = (\!|F(R,\ \mathbb{1})\ Q|\!)$	A map followed by a fold is a single fold — the mapped structure is never built.
naturality of α	$\alpha\ (T\ R) = F(R,\ T\ R)\ \alpha$	Building and then mapping is the same as mapping the parts and then building, so α is natural from $G\ R = F(R,\ T\ R)$ to T.
type relator	$(T\ R)^\circ = T\ (R^\circ)$	A datatype acts on relations, not only on maps — the map of the converse is the converse of the map.

Type functor fusion is the equality fusion (11.3a) applied to $T\ R$'s own defining algebra, whose side condition holds because F is a bifunctor — $F(R,\ \mathbb{1})\ F(\mathbb{1},\ (\!|Q|\!)) = F(R,\ (\!|Q|\!)) = F(\mathbb{1},\ (\!|Q|\!))\ F(R,\ \mathbb{1})$ — so it too needs only the initial algebra and the defining equation.

(13d)

$$\begin{tikzcd} F(A,\ T\ A) \arrow[r, "F(R,\ T\ R)"] \arrow[d, "\alpha"'] & F(B,\ T\ B) \arrow[d, "\alpha"] \\ T\ A \arrow[r, "T\ R"'] & T\ B \end{tikzcd}$$

It commutes strictly, and it is (13b) with the right vertical's two halves separated: push $F(R,\ \mathbb{1})$ up into the top row, where it meets $F(\mathbb{1},\ T\ R)$ as the one action $F(R,\ T\ R)$, and α is left as the vertical. In the string diagram that is the F_R bead sliding up past $T\ R$, and what stays put is α, one bead falling past $T\ R$ from the row above to the row below — which is what "α is natural" means.

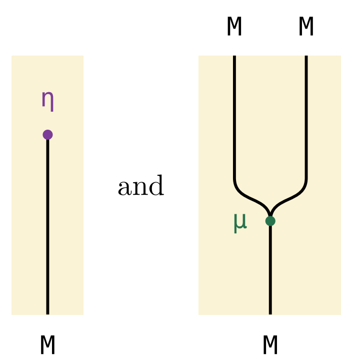
§14. Monad

(14a)

An endofunctor $M : \mathcal{A} \to \mathcal{A}$ with two natural transformations, the **unit** $\eta : Id \Rightarrow M$ and the **multiplication** $\mu : M{\circ}M \Rightarrow M$, subject to a unit law and an associativity law:

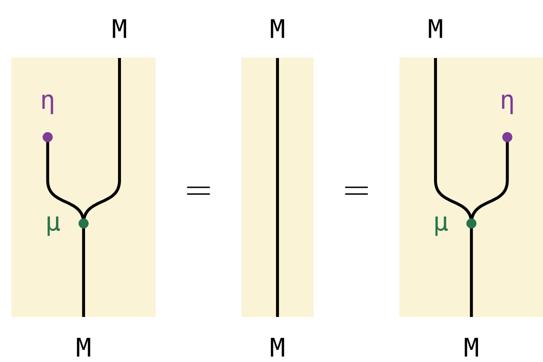
$$(\eta{\circ}M)\ \mu = \mathbb{1} = (M{\circ}\eta)\ \mu \qquad\qquad (\mu{\circ}M)\ \mu = (M{\circ}\mu)\ \mu$$

(14b)



In (14b) the identity functor is a REGION and not a wire, so $\eta : Id \Rightarrow M$ has nothing above it and the M wire simply begins at its bead — the book's lollipop. $\mu : M{\circ}M \Rightarrow M$ is the two M wires of $M{\circ}M$ merging into one, its tuning fork.

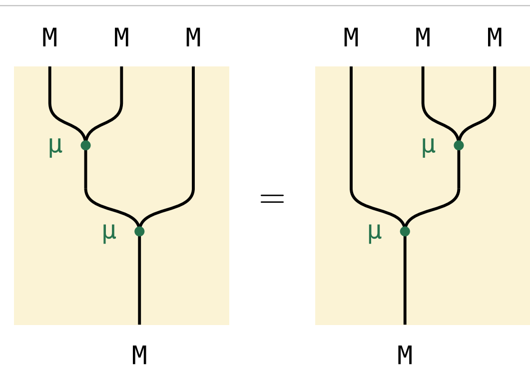
(14c)



$(\eta{\circ}M)\ \mu = \mathbb{1} = (M{\circ}\eta)\ \mu$

The lollipop is absorbed into the fork from either side and a bare wire is left, which is what $\mathbb{1}$ is in this calculus: nothing is drawn for it.

(14d)



$(\mu{\circ}M)\ \mu = (M{\circ}\mu)\ \mu$

The two nestings of the fork agree, so three Ms collapse to one whichever pair is multiplied first.

§14.1. Monad in $\mathcal{S}$et

(14.1a)

monad	η	μ	η on elements	μ on elements
`Id A = A`	𝟙	𝟙	`η A a = a`	`μ A a = a`
`Exc A = A + E`	ι_l	𝟙 ▿ ι_r	`η A a =` ι_l `a`	`μ A (`ι_l `y) = y` `μ A (`ι_r `e) =` ι_r `e`
`State A = (A × P)`^{P}	`curry 𝟙`	$(\mathtt{apply}_{A \times P})^{P}$	`η A a = λ x . (a, x)`	`μ A f = λ x . g y` where `(g, y) = f x`
`Pow A = [A]`	`{·}`	⋃	`η A a = {a}`	`μ A Xs = ⋃ Xs`

§14.2. The state monad

For a fixed set `P` of states, IntroString p. 67 defines

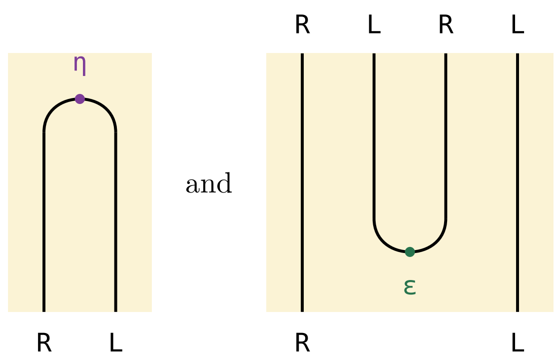
$$\mathtt{State\ A} = (\mathtt{A} \times \mathtt{P})^{\mathtt{P}} \qquad \eta\ \mathtt{A} = \mathtt{curry}\ (𝟙 : \mathtt{A} \times \mathtt{P} \to \mathtt{A} \times \mathtt{P}) \qquad \mu\ \mathtt{A} = (\mathtt{apply}_{\mathtt{A} \times \mathtt{P}})^{\mathtt{P}}$$

Both arrows come from the curry adjunction `L ⊣ R`, `L A = A × P` and `R B =` $\mathtt{B}^{\mathtt{P}}$ (IntroString p. 96):

- `State = R∘L`.
- `η` **is** the unit `Id ⇒ R∘L`, which is why it is the curried identity.
- the counit `ε : L∘R ⇒ Id` is `apply`.
- `μ = R∘ε∘L`, that counit with a functor on each side: in `μ A =` $(\mathtt{apply}_{\mathtt{A} \times \mathtt{P}})^{\mathtt{P}}$ the subscript `A × P` is `L A`, the functor INSIDE, and the outer $(-)^{\mathtt{P}}$ is `R`, the functor OUTSIDE.
- `⊣` composes nothing; it only names the left adjoint first.

Applicative order stands `R` left and `L` right in `State = R∘L`. The identity functor is a region, so a unit `Id ⇒ R∘L` has nothing above it and two wires below — one strand that OPENS the pair; a counit `L∘R ⇒ Id` is that strand upside down, CLOSING a pair that arrives from above.

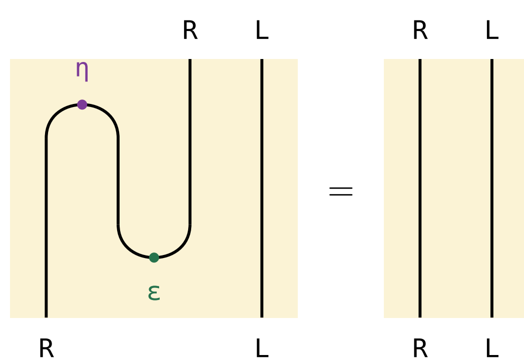
(14.2a)



In (14.2a), `State∘State` is the four wires `R L R L`, and `μ = R∘ε∘L` closes the middle pair — the `L∘R` — leaving the outer `R` and `L`, which is `State` again. The two wires that run past the turn untouched are the two functors composed on, `R` outside and `L` inside.

(14.2b)

$$\begin{tikzcd} \mathtt{State\ A} \arrow[r, "\eta\ (\mathtt{State\ A})"] \arrow[dr, "𝟙"'] & \mathtt{State\ (State\ A)} \arrow[d, "\mu\ \mathtt{A}"] \\ & \mathtt{State\ A} \end{tikzcd}$$



`(η∘State) μ = 𝟙`

(14.2b) is the zig-zag. `η∘State` opens a new `R L` left of the pair already there and `μ` closes the middle two — the new `L` against the old `R` — leaving the new `R` and the old `L`, which is `State` unchanged. Strip the old `L`, which never moves, and what zig-zags is the curry adjunction's snake equation `(η∘R)(R∘ε) = 𝟙` (IntroString p. 93): (14c) read at `State` is that triangle identity with `L` composed on the inside.

In Set it is beta-reduction. `η (State A) f = λ x . (f, x)` pairs the computation with the state, and `μ A` applies it back, `λ x . f x`, which is `f`.